```
      {
        "type": "h1",
        "props": {
          "children": "Hello"
        }
      },
      {
        "type": "p",
        "props": {
          "children": "World"
        }
      }
    ]
  }
}
```

At this stage, React builds a work-in-progress tree to prepare for the next phases but does not yet apply any changes to the real DOM.

The second phase is the pre-commit phase. Here, React prepares for atomic DOM updates by batching all changes together, ensuring the UI is updated consistently and avoiding any intermediate, partially-rendered states.

For class components, React invokes the `getSnapshotBeforeUpdate()` lifecycle method during this phase. Internally, React validates the work-in-progress tree, aligns it with any pending updates, and ensures consistency between the current tree and the next state before making real DOM changes. This preparation step is crucial for ensuring that updates are smooth and free of race conditions, particularly in asynchronous environments.

The third and final phase is the commit phase. Once reconciliation determines the minimal set of changes needed, React applies these updates to the real DOM synchronously and irreversibly.

During this phase, React updates the actual DOM, sets up DOM refs and layout effects, triggers the browser's paint process, and runs effect hooks such as `useEffect` and `useLayoutEffect`. For class components, React calls lifecycle methods like `componentDidMount()` and `componentDidUpdate()`.

React's batching mechanism ensures that only changed components are updated. This minimizes direct DOM manipulations, reduces layout